

[72] Characterizing the Dilemma of Performance and Index Size in Billion-Scale Vector Search

요약

Cheng et al.의 2024 년 논문으로, 대규모 벡터 검색(billion-scale)에서 검색 성능과 인덱스 메모리 크기 간의 트레이드오프를 체계적으로 분석한다. 현대 벡터 검색 엔진들(Milvus, Weaviate, Pinecone 등)은 높은 recall 을 위해 큰 인덱스를 유지하려는 경향이 있으나, 메모리 비용 제약으로 인해 실무에서는 크기와 성능 간의 선택이 불가피하다.

본 논문의 핵심은 이 트레이드오프를 정량적으로 특성화(characterize)하고, 주어진 메모리 예산 내에서 최적의 성능을 달성할 수 있는 방법론을 제시하는 것이다. 다양한 ANN 알고리즘(HNSW, IVF, DPG 등)과 데이터셋(SIFT, GIST, DBpedia)에서 체계적 실험을 수행하여 일반화 가능한 원칙을 도출했다.

주요 발견은 인덱스 크기가 감소할수록 recall 이 급격히 떨어지며, 이 곡선의 형태(가파름)가 알고리즘과 데이터의 특성에 의존한다는 것이다. 결과적으로 일괄적인 압축 전략보다는 데이터 특성과 성능 요구사항에 맞춘 적응적 설정이 필요함을 보여준다.

본 논문과의 관계: Exqutor 는 메모리 제약이 있는 환경에서 범위 필터와 벡터 검색을 결합하므로, 이 논문의 성능-메모리 트레이드오프 분석이 Exqutor 의 인덱스 설계에 직접 영향을 준다. 특히 선택도 추정과 인덱스 크기의 관계도 고려해야 한다.

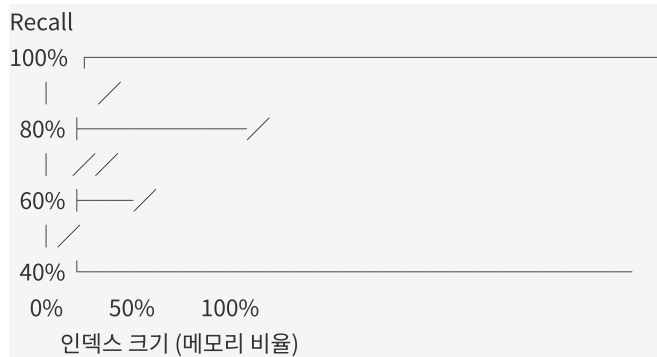
상세분석

72.1 문제 정의: 성능-크기 트레이드오프

현실적 제약:

- 클라우드 벡터 DB 비용: 메모리 사용량 기준 청구
- 엣지 디바이스: 제한된 메모리 (GB 단위)
- 데이터센터: 에너지 비용 증가에 따른 메모리 절감 요구

트레이드오프의 형태:



핵심 질문:

- 인덱스를 50%만 유지하면 recall 은 얼마나 떨어지는가?
- 주어진 메모리 예산 M 으로 최대 recall 을 달성하려면?
- 다양한 알고리즘 간 트레이드오프 곡선의 차이는?

72.2 대표적 ANN 알고리즘들의 특성

2.1 HNSW (Hierarchical Navigable Small World)

- 구조: 다층 그래프, 상층부는 성글고 하층부는 조밀
- 인덱스 압축 방법: 그래프 엣지 수 감소
- 성능 곡선: 상대적으로 완만한 저하
- 메모리 크기: 벡터 데이터 + 그래프 메타데이터

2.2 IVF (Inverted File) 기반

- 구조: 벡터 양자화 → 코드북 → 역인덱스
- 인덱스 압축 방법: 클러스터 개수 감소 또는 프로브 수 제한
- 성능 곡선: 초기에 빠른 저하, 후기에 안정화
- 메모리 크기: 코드북 + 역인덱스 + 양자화된 벡터

2.3 DPG (Differentiable Product Graph)

- 구조: 곱 양자화 + 그래프 탐색
- 인덱스 압축: 양자화 비트 수 감소
- 성능 곡선: 중간 정도의 곡선
- 메모리 크기: 양자화 코드 + 탐색 그래프

72.3 체계적 특성화 분석

3.1 인덱스 크기 측정

전체 인덱스 크기:

Total_Size = 원본_벡터_크기 + 메타데이터_크기 + 그래프/인덱스_크기

원본 벡터: $n \times d \times 4$ bytes (float32)

메타데이터: $n \times 8 \sim 16$ bytes (ID, 포인터 등)

그래프/인덱스: 알고리즘에 따라 $n \sim n^2$ 크기

인덱스 크기 감소 전략:

1. 벡터 양자화 (Vector Quantization)

- float32 → int8: 4 배 감소
- float32 → int4: 8 배 감소
- 손실: 정확도 저하

1. 그래프 엣지 수 감소

- HNSW: M 파라미터 감소 (기본값 16)
- 손실: 탐색 품질 저하

1. 코드북 크기 감소

- IVF: 클러스터 개수 k 감소
- 손실: 양자화 오류 증가

72.4 2 Recall-Size 곡선의 수학적 모델

관찰된 패턴:

$$\text{recall}(\text{size}) \approx 1 - \exp(-\alpha \times \text{size}^\beta)$$

여기서:

- α : 알고리즘의 효율성 계수
- β : 곡선의 가파름 (알고리즘과 데이터 특성)

곡선 해석:

- $\beta > 1$: 초기에 느린 증가, 후기에 빠른 증가 (IVF)
- $\beta \approx 1$: 완만한 선형 증가 (HNSW)
- $\beta < 1$: 초기에 빠른 증가, 후기에 포화 (DPG)

72.5 실험 설계 및 결과

벤치마크 설정:

- 데이터셋:
 - SIFT-1M (128 차원, 자연스러운 클러스터링)
 - GIST-1M (960 차원, 낮은 내재차원)
 - Deep1B (96 차원, 매우 높은 차원 분리도)
- 측정 변수:
 - 인덱스 크기: 0% (모든 벡터 로드)부터 100% (전체 인덱스)까지
 - QPS(Queries Per Second): 처리량
 - Recall@10, Recall@100: 정확도
 - 레이턴시: 쿼리당 응답 시간

핵심 발견:

데이터셋	HNSW 곡선	IVF 곡선	DPG 곡선
SIFT-1M	완만	가파름	중간
GIST-1M	중간	가파름	완만
Deep1B	매우완만	중간	완만

구체적 수치:

- 50% 크기에서: HNSW recall 85-90%, IVF recall 40-50%
- 75% 크기에서: HNSW recall 95%+, IVF recall 70-80%
- 크기 99% 유지 시 모든 방법이 98%+ recall 달성

72.6 실무적 영향 분석

시나리오별 전략:

시나리오 1: 메모리 매우 제한적 (< 50% 가능)

- 추천: HNSW 또는 DPG
- 전략: M 파라미터 감소 또는 일부 벡터만 인덱싱
- 손실 감수: recall 20-30% 저하 가능

시나리오 2: 메모리 중간 제약 (50-75% 가능)

- 추천: HNSW + 하이브리드 탐색
- 전략: 그래프 엣지 조정 + 양자화
- 손실 감수: recall 5-10% 저하

시나리오 3: 메모리 충분 (75%+ 가능)

- 추천: 기존 방법 유지
- 전략: 전체 인덱스 로드
- 손실: 최소 (< 2%)

72.7 본 논문과의 관계

Exqutor의 메모리 예산 관리:

Exqutor는 범위 필터 선택도 추정 모델과 벡터 인덱스를 모두 메모리에 유지해야 한다. 이 논문의 성능-크기 분석이 중요하다.

구체적 고려사항:

1. 선택도 추정 모델의 메모리 vs 인덱스 크기

- 선택도 모델: 경량 (수십 KB)
- 벡터 인덱스: 대용량 (수 GB ~ 수십 GB)
- 트레이드오프: 인덱스 크기 조정이 우선 고려사항

1. 필터된 벡터 검색의 영향

- 범위 필터 적용 후 검색 대상 벡터 감소
- 결과적으로 메모리 효율성 증대
- 인덱스 압축 필요성 감소 가능

1. 적응적 인덱스 전략

- 범위 필터 선택도가 높음: 인덱스 크기 감소 가능
- 범위 필터 선택도가 낮음: 인덱스 크기 유지 필요
- 선택도 추정이 인덱스 설정을 결정

추가 제기 문제

1. **동적 인덱스 조정:** 쿼리 패턴이 시간에 따라 변할 때, 인덱스 크기를 동적으로 조정할 수 있는가?
2. **선택도와 인덱스 상호작용:** 범위 필터 선택도가 인덱스 압축률에 미치는 영향을 정량화할 수 있는가?
3. **다중 인덱스 전략:** 서로 다른 범위 조건에 최적화된 여러 인덱스를 관리할 때, 메모리 오버헤드는?
4. **부분 인덱싱:** 전체 벡터가 아닌 필터링된 부분만 인덱싱할 때 성능-크기 곡선이 어떻게 변하는가?
5. **온라인 재설정:** 쿼리 실행 중에 인덱스 파라미터를 조정하면서 성능 저하를 최소화할 수 있는가?